

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_02.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Use conditional structures `if/elif/else`
- Write complex conditions with `and`, `or`, `not`
- Handle input errors with `try/except`
- Validate user-entered data
- Apply conditions to concrete problems (classification, validation)

Lab Plan

Phase	Content	Duration
Welcome	Review of conditions	10 min
Exercise 1	Parity check	15 min
Exercise 2	Grade-based distinction	15 min
Exercise 3	Age category	15 min
Exercise 4	Grade converter (letters → numbers)	20 min
Exercise 5	Password validation	15 min

Prerequisites

- Lab 01 completed (variables, `print()`, `input()`, type conversions)
- Python 3 installed and working

Exercise 1 — Parity check (15 min)

Problem statement: Create a script `parite.py` that:

1. Asks the user for an integer
2. Displays whether the number is even or odd
3. Also displays whether the number is divisible by 3 and by 5

Expected output:

```
Enter a number : 15
15 is odd
15 is divisible by 3 and 5
```

Solution:

```
try:
    n = int(input("Enter a number : "))
    if n % 2 == 0:
        print(f"{n} is even")
    else:
        print(f"{n} is odd")
    if n % 3 == 0 and n % 5 == 0:
        print(f"{n} is divisible by 3 and 5")
    elif n % 3 == 0:
        print(f"{n} is divisible by 3")
    elif n % 5 == 0:
        print(f"{n} is divisible by 5")
except ValueError:
    print("Please enter a valid integer.")
```

Exercise 2 — Grade-based distinction (15 min)

Problem statement: Create a script mention.py that asks for a baccalaureate average (0–20) and displays the distinction:

- ≥ 16 : "Very Good"
- ≥ 14 : "Good"
- ≥ 12 : "Fairly Good"
- ≥ 10 : "Passable"
- < 10 : "Not admitted"

Handle the case where the grade is not between 0 and 20.

Expected output:

```
Baccalaureate average (0–20) : 15
Good
```

Solution:

```
try:
    average = float(input("Baccalaureate average (0–20) : "))
    if average < 0 or average > 20:
        print("The average must be between 0 and 20.")
    elif average >= 16:
        print("Very Good")
    elif average >= 14:
        print("Good")
    elif average >= 12:
        print("Fairly Good")
    elif average >= 10:
        print("Passable")
    else:
        print("Not admitted")
except ValueError:
    print("Error: please enter a valid number.")
```

Exercise 3 — Age category (15 min)

Problem statement: Create a script `categorie.py` that asks for a person's age and displays the age range:

- 0-2 years: Baby
- 3-12 years: Child
- 13-17 years: Adolescent
- 18-59 years: Adult
- 60+ years: Senior
- < 0: Invalid age

Expected output:

```
Age : 25
Category : Adult
```

Solution:

```
try:
    age = int(input("Age : "))
    if age < 0:
        print("Invalid age.")
    elif age <= 2:
        print("Category : Baby")
    elif age <= 12:
        print("Category : Child")
    elif age <= 17:
        print("Category : Adolescent")
    elif age <= 59:
        print("Category : Adult")
    else:
        print("Category : Senior")
except ValueError:
    print("Please enter a valid age.")
```

Exercise 4 — Grade converter (letters → numbers) (20 min)

Problem statement: Create a script `noteslettres.py` that converts a letter grade to a numerical grade out of 20 according to the table:

- A+ → 19, A → 17, A- → 16
- B+ → 15, B → 14, B- → 13
- C+ → 12, C → 11, C- → 10
- D+ → 9, D → 8, D- → 7
- E → 5, F → 2

The user enters a letter (with or without + or -) and gets the grade out of 20.

Expected output:

```
Letter grade (A+, B, C-, etc.) : B+
Numerical grade : 15/20
```

Solution:

```
grade = input("Letter grade (A+, B, C-, etc.) : ").upper()

conversion = {
```

```

"A+": 19, "A": 17, "A-": 16,
"B+": 15, "B": 14, "B-": 13,
"C+": 12, "C": 11, "C-": 10,
"D+": 9, "D": 8, "D-": 7,
"E": 5, "F": 2
}

if grade in conversion:
    print(f"Numerical grade : {conversion[grade]}/20")
else:
    print("Invalid letter grade.")

```

Exercise 5 — Password validation (15 min)

Problem statement: Create a script `mdp_valide.py` that validates a password according to these criteria:

- At least 8 characters
- Contains at least one uppercase letter
- Contains at least one digit
- Contains at least one symbol among `!@#$$%`

Hint: Use the methods `isupper()`, `isdigit()`, and `any()`.

Expected output:

```

Password : Python@2025
Valid password.

```

Solution:

```

pwd = input("Password : ")

length_ok = len(pwd) >= 8
upper_ok = any(c.isupper() for c in pwd)
digit_ok = any(c.isdigit() for c in pwd)
symbol_ok = any(c in "!@#$$%" for c in pwd)

if length_ok and upper_ok and digit_ok and symbol_ok:
    print("Valid password.")
else:
    print("Invalid password.")
    if not length_ok:
        print(" - At least 8 characters")
    if not upper_ok:
        print(" - At least one uppercase letter")
    if not digit_ok:
        print(" - At least one digit")
    if not symbol_ok:
        print(" - At least one symbol (!@#$$%)")

```

Logic of nested conditions (grade converter):

```

flowchart TD
    A[Start] --> B[Enter letter grade]
    B --> C[Normalize to uppercase]
    C --> D{Grade in\ndictionary?}
    D -->|Yes| E[Get numerical value]
    E --> F[Display grade/20]
    D -->|No| G[Display error]
    F --> H[End]
    G --> H

```

G --> H

General if/elif/else structure:

```

flowchart TD
  A[Condition 1] -->|True| B[Block 1]
  A -->|False| C[Condition 2]
  C -->|True| D[Block 2]
  C -->|False| E[Condition 3]
  E -->|True| F[Block 3]
  E -->|False| G[else: Final block]

```

Self-assessment quiz

1. What does the expression `not (10 > 5)` return?

- a) True
- b) False ✓
- c) None

2. What is the output of `print("A" if 3 > 5 else "B")`?

- a) A
- b) B ✓
- c) True

3. What is the correct operator for testing equality in Python?

- a) =
- b) == ✓
- c) !=

4. In an `elif`, how many `else` can we have?

- a) 0 or 1 ✓
- b) As many as we want
- c) At least 1

5. What does `try/except` do?

- a) Repeats a block of code
- b) Catches and handles errors ✓
- c) Defines a function

Grading Rubric Lab 02

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
Conditional syntax	20%	if/elif/else incorrect	Partial logical order	Correct structure
Logical operators (and/or/not)	20%	Does not use operators	Partial use	Correct use
Error handling (try/except)	20%	No handling	Partial handling	Complete handling

Dictionary and conversion	20%	No dictionary	Incorrect use	Correct use
Input validation	20%	No validation	Minimal validation	Robust validation

Session schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome and conditions review	10 min	10 min
Exercise 1 — Parity	15 min	25 min
Exercise 2 — Grade-based distinction	15 min	40 min
Exercise 3 — Age category	15 min	55 min
Exercise 4 — Grade converter	20 min	75 min
Exercise 5 — Password validation	15 min	90 min

Pedagogical Note ENS

Teaching transposition

This lab introduces conditional structures, the cornerstone of algorithmic logic. For future teachers, each exercise is chosen for its potential for reuse in class: parity check (ex. 1) is a classic of middle school algorithmics; the distinction system (ex. 2) anchors computer science in everyday school life; age category (ex. 3) allows working with intervals and boundaries; the letter-to-number grade converter (ex. 4) is a direct application of the Python dictionary and connects to the school assessment system.

Trainer advice

Emphasize the `if/elif/else` structure and the difference between `=` (assignment) and `==` (comparison), a very common confusion. Use the comparison operators table as a cheat sheet. For high school students, transform exercise 5 into a running project: a semester average calculator with coefficients.

Extension

Add a loop to repeat the average calculation for several students, then export the results to a CSV file. This extension prepares Lab 07 on file management.